

TIME SERIES AND FORECASTING: AVOCADO PRICE PREDICTION

Vivian Jerono

2026-07-28

```
# Install packages (uncomment to run once)
# install.packages(c(
#   "tidyverse", "lubridate", "caret", "randomForest",
#   "rpart", "rpart.plot", "corrplot", "gridExtra",
#   "knitr", "kableExtra", "forecast", "prophet",
#   "tseries", "zoo", "ggplot2", "plotly", "shiny"
# ))
```

```
library(tidyverse)
```

```
## Warning: package 'tidyverse' was built under R version 4.5.3
```

```
## Warning: package 'ggplot2' was built under R version 4.5.3
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.2.0    ✓ readr      2.1.6
## ✓ forcats    1.0.1    ✓ stringr    1.6.0
## ✓ ggplot2    4.0.3    ✓ tibble     3.3.1
## ✓ lubridate  1.9.5    ✓ tidyr      1.3.2
## ✓ purrr      1.2.1
## — Conflicts — tidyverse_conflicts() —
## ✗ dplyr::filter() masks stats::filter()
## ✗ dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(lubridate)
```

```
library(caret)
```

```
## Loading required package: lattice
##
## Attaching package: 'caret'
##
## The following object is masked from 'package:purrr':
##
##   lift
```

```
library(randomForest)
```

```
## Warning: package 'randomForest' was built under R version 4.5.3
```

```
## randomForest 4.7-1.2
## Type rfNews() to see new features/changes/bug fixes.
##
## Attaching package: 'randomForest'
##
## The following object is masked from 'package:dplyr':
##
##   combine
##
## The following object is masked from 'package:ggplot2':
##
##   margin
```

```
library(rpart)
```

```
## Warning: package 'rpart' was built under R version 4.5.3
```

```
library(rpart.plot)
```

```
## Warning: package 'rpart.plot' was built under R version 4.5.3
```

```
library(corrplot)
```

```
## Warning: package 'corrplot' was built under R version 4.5.3
```

```
## corrplot 0.95 loaded
```

```
library(gridExtra)
```

```
##
## Attaching package: 'gridExtra'
##
## The following object is masked from 'package:randomForest':
##
##   combine
##
## The following object is masked from 'package:dplyr':
##
##   combine
```

```
library(knitr)
library(kableExtra)
```

```
## Warning: package 'kableExtra' was built under R version 4.5.3
```

```
##
## Attaching package: 'kableExtra'
##
## The following object is masked from 'package:dplyr':
##
##   group_rows
```

```
library(forecast)
```

```
## Registered S3 method overwritten by 'quantmod':
##   method      from
##   as.zoo.data.frame zoo
```

```
library(prophet)
```

```
## Warning: package 'prophet' was built under R version 4.5.3
```

```
## Loading required package: Rcpp
## Loading required package: rlang
##
## Attaching package: 'rlang'
##
## The following objects are masked from 'package:purrr':
##
##   flatten, flatten_chr, flatten_dbl, flatten_int, flatten_lgl,
##   flatten_raw, invoke, splice
```

```
library(tseries)
library(zoo)
```

```
##
## Attaching package: 'zoo'
##
## The following objects are masked from 'package:base':
##
##   as.Date, as.Date.numeric
```

```
library(plotly)
```

```
##
## Attaching package: 'plotly'
##
## The following object is masked from 'package:ggplot2':
##
##   last_plot
##
## The following object is masked from 'package:stats':
##
##   filter
##
## The following object is masked from 'package:graphics':
##
##   layout
```

```
library(shiny)
```

```
## Warning: package 'shiny' was built under R version 4.5.3
```

```
# Load the dataset
avocado <- read.csv("C:/Users/Admin/Downloads/avocado.csv/avocado.csv")
print(head(avocado, 5))
```

```
##      X      Date AveragePrice Total.Volume      X4046      X4225      X4770 Total.Bags
## 1 51 1/4/2015          1.22    40873.28    2819.50    28287.42    49.90    9716.46
## 2 51 1/4/2015          1.00   435021.49  364302.39    23821.16    82.15   46815.79
## 3 51 1/4/2015          1.08   788025.06  53987.31  552906.04  39995.03  141136.68
## 4 51 1/4/2015          1.01    80034.32   44562.12   24964.23   2752.35    7755.62
## 5 51 1/4/2015          1.02   491738.00   7193.87  396752.18   128.82   87663.13
##      Small.Bags Large.Bags XLarge.Bags      type year      region
## 1      9186.93     529.53           0 conventional 2015      Albany
## 2     16707.15    30108.64           0 conventional 2015      Atlanta
## 3    137146.07     3990.61           0 conventional 2015 BaltimoreWashington
## 4      6064.30     1691.32           0 conventional 2015      Boise
## 5     87406.84      256.29           0 conventional 2015      Boston
```

```
# Dataset overview
cat("===== AVOCADO DATASET OVERVIEW =====\n")
```

```
## ===== AVOCADO DATASET OVERVIEW =====
```

```
cat("Dataset dimensions:", dim(avocado), "\n")
```

```
## Dataset dimensions: 18249 14
```

```
cat("Number of rows:", nrow(avocado), "\n")
```

```
## Number of rows: 18249
```

```
cat("Number of columns:", ncol(avocado), "\n\n")
```

```
## Number of columns: 14
```

```
cat("\nColumn names:\n")
```

```
##
## Column names:
```

```
print(names(avocado))
```

```
## [1] "X"           "Date"         "AveragePrice" "Total.Volume" "X4046"
## [6] "X4225"       "X4770"        "Total.Bags"   "Small.Bags"   "Large.Bags"
## [11] "XLarge.Bags" "type"         "year"         "region"
```

```
cat("\nData structure:\n")
```

```
##
## Data structure:
```

```
str(avocado)
```

```
## 'data.frame': 18249 obs. of 14 variables:
## $ X : int 51 51 51 51 51 51 51 51 51 51 ...
## $ Date : chr "1/4/2015" "1/4/2015" "1/4/2015" "1/4/2015" ...
## $ AveragePrice: num 1.22 1 1.08 1.01 1.02 1.4 0.93 1.19 1.11 0.88 ...
## $ Total.Volume: num 40873 435021 788025 80034 491738 ...
## $ X4046 : num 2820 364302 53987 44562 7194 ...
## $ X4225 : num 28287 23821 552906 24964 396752 ...
## $ X4770 : num 49.9 82.2 39995 2752.3 128.8 ...
## $ Total.Bags : num 9716 46816 141137 7756 87663 ...
## $ Small.Bags : num 9187 16707 137146 6064 87407 ...
## $ Large.Bags : num 530 30109 3991 1691 256 ...
## $ XLarge.Bags : num 0 0 0 0 0 ...
## $ type : chr "conventional" "conventional" "conventional" "conventional" ...
## $ year : int 2015 2015 2015 2015 2015 2015 2015 2015 2015 2015 ...
## $ region : chr "Albany" "Atlanta" "BaltimoreWashington" "Boise" ...
```

```
cat("\nSummary statistics:\n")
```

```
##
## Summary statistics:
```

```
summary(avocado)
```

```
##           X           Date      AveragePrice      Total.Volume
## Min.      : 0.00      Length:18249      Min.      :0.440      Min.      :      85
## 1st Qu.:10.00      Class :character      1st Qu.:1.100      1st Qu.:   10839
## Median :24.00      Mode  :character      Median :1.370      Median :   107377
## Mean      :24.23                                Mean      :1.406      Mean      :   850644
## 3rd Qu.:38.00                                3rd Qu.:1.660      3rd Qu.:   432962
## Max.       :52.00                                Max.       :3.250      Max.       :62505647
##           X4046           X4225           X4770           Total.Bags
## Min.      :0.000e+00      Min.      :      0      Min.      :      0      Min.      :      0
## 1st Qu.:8.541e+02      1st Qu.:   3009      1st Qu.:      0      1st Qu.:   5089
## Median :8.645e+03      Median :   29061      Median :    185      Median :   39744
## Mean      :2.930e+05      Mean      :  295155      Mean      : 22840      Mean      :  239639
## 3rd Qu.:1.110e+05      3rd Qu.:  150207      3rd Qu.:   6243      3rd Qu.:  110783
## Max.      :2.274e+07      Max.      :20470573      Max.      :2546439      Max.      :19373134
##      Small.Bags      Large.Bags      XLarge.Bags      type
## Min.      :      0      Min.      :      0.0      Min.      :      0.0      Length:18249
## 1st Qu.:   2849      1st Qu.:   127.5      1st Qu.:      0.0      Class :character
## Median :   26363      Median :   2647.7      Median :      0.0      Mode  :character
## Mean      :  182195      Mean      :  54338.1      Mean      :  3106.4
## 3rd Qu.:   83338      3rd Qu.:  22029.2      3rd Qu.:   132.5
## Max.      :13384587      Max.      :5719096.6      Max.      :551693.7
##           year           region
## Min.      :2015      Length:18249
## 1st Qu.:2015      Class :character
## Median :2016      Mode  :character
## Mean      :2016
## 3rd Qu.:2017
## Max.      :2018
```

```
cat("\nMissing values per column:\n")
```

```
##
## Missing values per column:
```

```
missing_count <- colSums(is.na(avocado))
print(missing_count[missing_count > 0])
```

```
## named numeric(0)
```

DATA CLEANING

```
# Remove unnecessary first column
avocado <- avocado %>% select(-X)

# Convert Date to proper format
avocado <- avocado %>%
  mutate(
    Date = as.Date(Date, format = "%m/%d/%Y"),
    year = year(Date),
    month = month(Date, label = TRUE),
    quarter = quarter(Date),
    day_of_week = wday(Date, label = TRUE)
  )

# Check for duplicates
duplicates <- sum(duplicated(avocado))
cat("\nNumber of duplicate rows:", duplicates, "\n")
```

```
##
## Number of duplicate rows: 0
```

```
# Check for missing values
cat("\nMissing values after cleaning:\n")
```

```
##
## Missing values after cleaning:
```

```
print(colSums(is.na(avocado)))
```

```
##      Date AveragePrice Total.Volume      X4046      X4225      X4770
##      0           0           0           0           0           0
## Total.Bags Small.Bags Large.Bags XLarge.Bags      type      year
##      0           0           0           0           0           0
##      region      month      quarter day_of_week
##      0           0           0           0
```

```
# Verify date conversion
cat("Date class:", class(avocado$Date), "\n")
```

```
## Date class: Date
```

```
cat("Date range:", min(avocado$Date), "to", max(avocado$Date), "\n")
```

```
## Date range: 16439 to 17615
```

```
cat("Sample dates:\n")
```

```
## Sample dates:
```

```
print(head(avocado$Date, 10))
```

```
## [1] "2015-01-04" "2015-01-04" "2015-01-04" "2015-01-04" "2015-01-04"
## [6] "2015-01-04" "2015-01-04" "2015-01-04" "2015-01-04" "2015-01-04"
```

EXPLORATORY DATA ANALYSIS

```
# Target variable distribution
cat("\n===== TARGET VARIABLE ANALYSIS =====\n")
```

```
##
## ===== TARGET VARIABLE ANALYSIS =====
```

```
cat("AveragePrice Summary Statistics:\n")
```

```
## AveragePrice Summary Statistics:
```

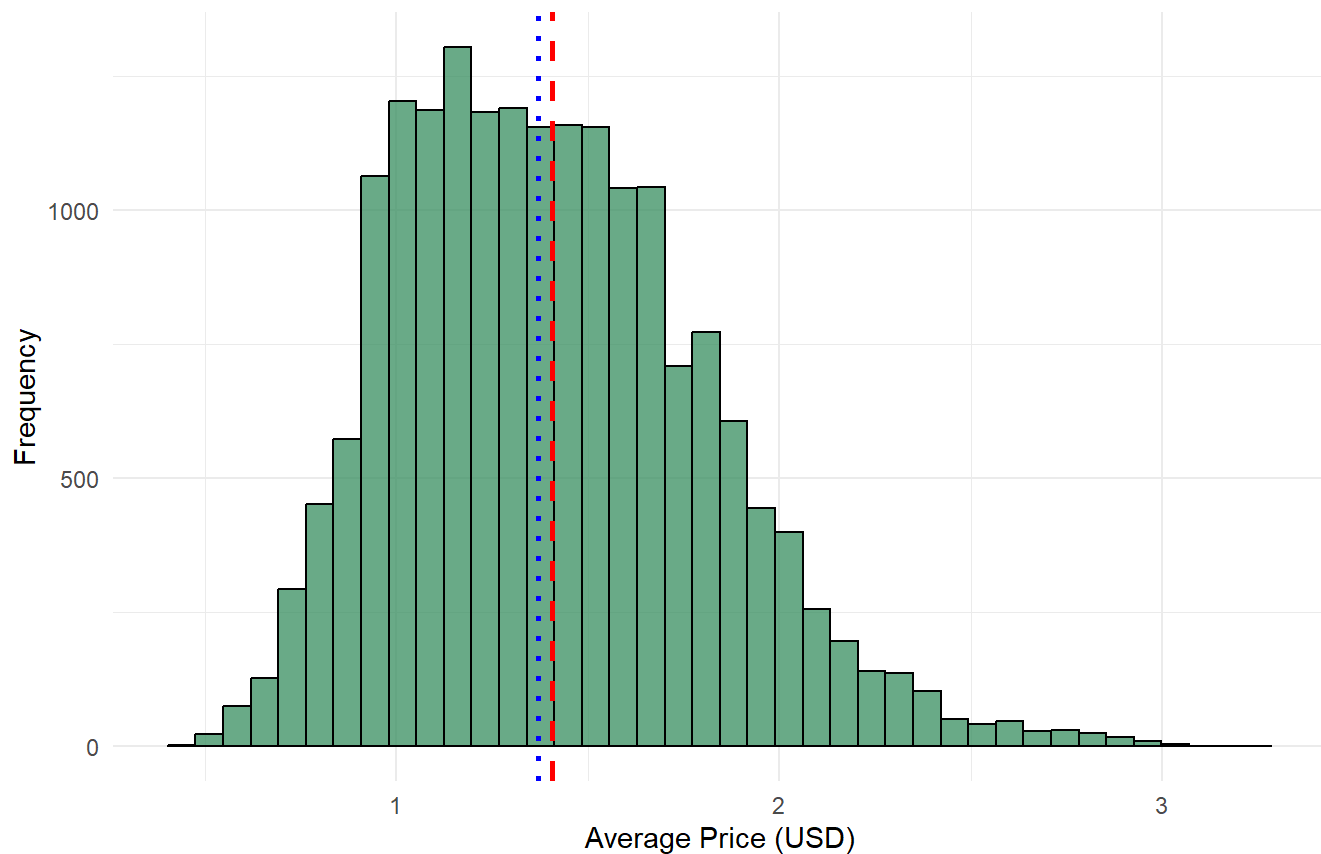
```
summary_stats <- summary(avocado$AveragePrice)
print(summary_stats)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##    0.440  1.100   1.370   1.406   1.660   3.250
```

```
# Price distribution histogram
p1 <- ggplot(avocado, aes(x = AveragePrice)) +
  geom_histogram(bins = 40, fill = "#2E8B57", color = "black", alpha = 0.7) +
  geom_vline(xintercept = mean(avocado$AveragePrice), color = "red", linetype = "dashed", linewidth = 1) +
  geom_vline(xintercept = median(avocado$AveragePrice), color = "blue", linetype = "dotted", linewidth = 1) +
  labs(
    title = "Distribution of Avocado Prices",
    subtitle = "Red line = Mean, Blue line = Median",
    x = "Average Price (USD)",
    y = "Frequency"
  ) +
  theme_minimal()
print(p1)
```


Distribution of Avocado Prices

Red line = Mean, Blue line = Median



```
# Price by Type
p2 <- ggplot(avocado, aes(x = type, y = AveragePrice, fill = type)) +
  geom_boxplot(outlier.shape = NA) +
  geom_jitter(width = 0.2, alpha = 0.2, size = 0.5) +
  labs(
    title = "Avocado Prices by Type",
    x = "Type",
    y = "Average Price (USD)"
  ) +
  scale_fill_manual(values = c("#FF6B6B", "#4ECDC4")) +
  theme_minimal()
print(p2)
```



```
# Price trend over time
p3 <- avocado %>%
  group_by(Date) %>%
  summarise(avg_price = mean(AveragePrice)) %>%
  ggplot(aes(x = Date, y = avg_price)) +
  geom_line(color = "#2E8B57", linewidth = 0.8) +
  geom_smooth(method = "loess", color = "red", se = FALSE) +
  labs(
    title = "Avocado Price Trend Over Time",
    x = "Date",
    y = "Average Price (USD)"
  ) +
  theme_minimal()
print(p3)
```

```
## `geom_smooth()` using formula = 'y ~ x'
```

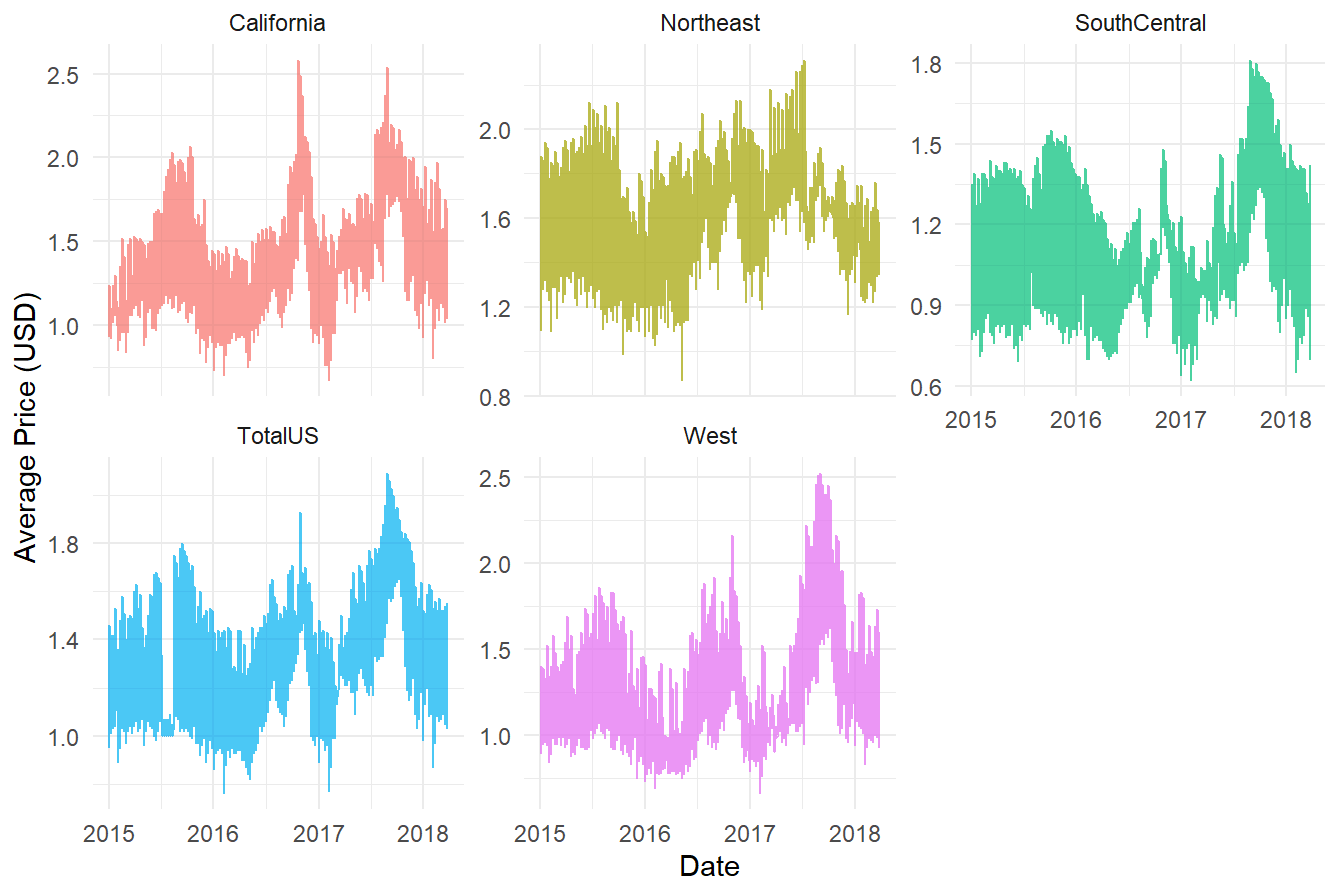
Avocado Price Trend Over Time



```
# Time series by region (top 5)
top_regions <- avocado %>%
  group_by(region) %>%
  summarise(total_volume = sum(Total.Volume)) %>%
  arrange(desc(total_volume)) %>%
  head(5) %>%
  pull(region)

p4 <- avocado %>%
  filter(region %in% top_regions) %>%
  ggplot(aes(x = Date, y = AveragePrice, color = region)) +
  geom_line(alpha = 0.7) +
  facet_wrap(~region, scales = "free_y") +
  labs(
    title = "Price Trends - Top 5 Regions",
    x = "Date",
    y = "Average Price (USD)"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
print(p4)
```

Price Trends - Top 5 Regions



TIME SERIES INITIAL ANALYSIS

```
# Select California for initial time series analysis
ca_data <- avocado %>%
  filter(region == "California") %>%
  arrange(Date)

ca_ts <- ts(
  ca_data$AveragePrice,
  start = c(2015, 1),
  end = c(2018, 12),
  frequency = 52
)

cat("California time series:\n")
```

```
## California time series:
```

```
cat("  Start:", start(ca_ts), "\n")
```

```
##    Start: 2015 1
```

```
cat("  End:", end(ca_ts), "\n")
```

```
## End: 2018 12
```

```
cat(" Frequency:", frequency(ca_ts), "weeks/year\n")
```

```
## Frequency: 52 weeks/year
```

```
cat(" Observations:", length(ca_ts), "\n")
```

```
## Observations: 168
```

```
# Check for seasonality
ca_by_month <- ca_data %>%
  mutate(month = month(Date, label = TRUE)) %>%
  group_by(month) %>%
  summarise(avg_price = mean(AveragePrice))

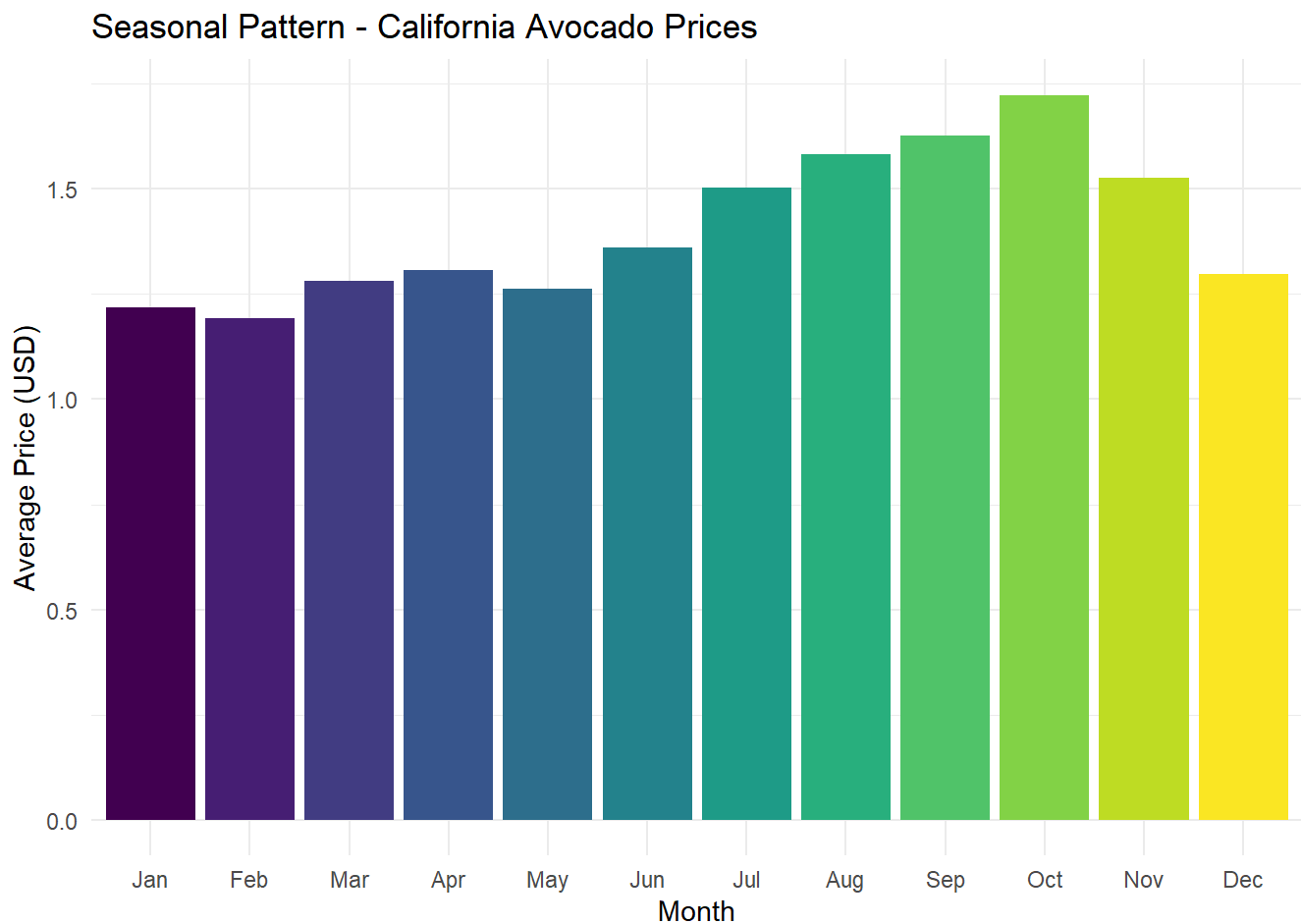
cat("\nAverage price by month (California):\n")
```

```
##
## Average price by month (California):
```

```
print(ca_by_month)
```

```
## # A tibble: 12 × 2
##   month avg_price
##   <ord>     <dbl>
## 1 Jan       1.22
## 2 Feb       1.19
## 3 Mar       1.28
## 4 Apr       1.31
## 5 May       1.26
## 6 Jun       1.36
## 7 Jul       1.50
## 8 Aug       1.58
## 9 Sep       1.63
## 10 Oct      1.72
## 11 Nov      1.53
## 12 Dec      1.30
```

```
# Monthly pattern plot
p5 <- ca_by_month %>%
  ggplot(aes(x = month, y = avg_price, fill = month)) +
  geom_col() +
  labs(
    title = "Seasonal Pattern - California Avocado Prices",
    x = "Month",
    y = "Average Price (USD)"
  ) +
  theme_minimal() +
  theme(legend.position = "none")
print(p5)
```



```
cat("=== CALIFORNIA DATA ===\n")
```

```
## === CALIFORNIA DATA ===
```

```
cat("Date range:", format(min(ca_data$Date), "%Y-%m-%d"), "to",
    format(max(ca_data$Date), "%Y-%m-%d"), "\n")
```

```
## Date range: 2015-01-04 to 2018-03-25
```

```
cat("Number of weeks:", nrow(ca_data), "\n")
```

```
## Number of weeks: 338
```

```
cat("\n=== SUMMARY STATISTICS ===\n")
```

```
##  
## === SUMMARY STATISTICS ===
```

```
cat("Data period:", format(min(ca_data$Date), "%Y-%m-%d"), "to",  
    format(max(ca_data$Date), "%Y-%m-%d"), "\n")
```

```
## Data period: 2015-01-04 to 2018-03-25
```

```
cat("Number of weeks:", nrow(ca_data), "\n")
```

```
## Number of weeks: 338
```

```
cat("Average price:", round(mean(ca_data$AveragePrice), 3), "\n")
```

```
## Average price: 1.395
```

```
cat("Price range:", round(min(ca_data$AveragePrice), 3),  
    "to", round(max(ca_data$AveragePrice), 3), "\n")
```

```
## Price range: 0.67 to 2.58
```

```
cat("Standard deviation:", round(sd(ca_data$AveragePrice), 3), "\n")
```

```
## Standard deviation: 0.391
```

```
cat("\n===== OUTLIER DETECTION =====\n")
```

```
##  
## ===== OUTLIER DETECTION =====
```

```
Q1 <- quantile(avocado$AveragePrice, 0.25)
Q3 <- quantile(avocado$AveragePrice, 0.75)
IQR <- Q3 - Q1
lower_bound <- Q1 - 1.5 * IQR
upper_bound <- Q3 + 1.5 * IQR

outliers_count <- sum(avocado$AveragePrice < lower_bound | avocado$AveragePrice > upper_bound)
cat("Outliers detected:", outliers_count, "(", round(outliers_count/nrow(avocado)*100, 2), "%)
\n")
```

```
## Outliers detected: 209 ( 1.15 %)
```

```
# Flag outliers but KEEP them
avocado <- avocado %>%
  mutate(
    is_outlier = ifelse(
      AveragePrice < lower_bound | AveragePrice > upper_bound,
      1, 0
    )
  )
```

```
cat("\n===== FEATURE ENGINEERING =====\n")
```

```
##
## ===== FEATURE ENGINEERING =====
```



```

# Create aggregate features
avocado <- avocado %>%
  mutate(
    # Bag percentages
    Bags_Percentage = (Total.Bags / Total.Volume) * 100,
    Small_Bags_Pct = (Small.Bags / Total.Bags) * 100,
    Large_Bags_Pct = (Large.Bags / Total.Bags) * 100,
    XLarge_Bags_Pct = (XLarge.Bags / Total.Bags) * 100,

    # Handle division by zero
    Small_Bags_Pct = ifelse(is.na(Small_Bags_Pct), 0, Small_Bags_Pct),
    Large_Bags_Pct = ifelse(is.na(Large_Bags_Pct), 0, Large_Bags_Pct),
    XLarge_Bags_Pct = ifelse(is.na(XLarge_Bags_Pct), 0, XLarge_Bags_Pct),

    # Total PLU volume
    Total_PLU_Volume = `X4046` + `X4225` + `X4770`
  )

# Create seasonal features
avocado <- avocado %>%
  mutate(
    season = case_when(
      month %in% c("Dec", "Jan", "Feb") ~ "Winter",
      month %in% c("Mar", "Apr", "May") ~ "Spring",
      month %in% c("Jun", "Jul", "Aug") ~ "Summer",
      month %in% c("Sep", "Oct", "Nov") ~ "Fall"
    ),
    is_holiday_season = ifelse(
      month %in% c("Nov", "Dec") | (month == "Jan" & day(Date) <= 7),
      TRUE, FALSE
    ),
    is_summer = ifelse(month %in% c("Jun", "Jul", "Aug"), TRUE, FALSE)
  )

# TIME SERIES FEATURES - LAG VARIABLES
cat("\nCreating time series lag features...\n")

```

```

##
## Creating time series lag features...

```

```

avocado <- avocado %>%
  arrange(region, Date) %>%
  group_by(region) %>%
  mutate(
    # Price lags
    price_lag_1 = lag(AveragePrice, 1),
    price_lag_2 = lag(AveragePrice, 2),
    price_lag_3 = lag(AveragePrice, 3),
    price_lag_4 = lag(AveragePrice, 4),
    price_lag_8 = lag(AveragePrice, 8),

    # Volume lags
    volume_lag_1 = lag(Total.Volume, 1),
    volume_lag_2 = lag(Total.Volume, 2),

    # Price changes
    price_change_1 = AveragePrice - price_lag_1,
    price_change_2 = AveragePrice - price_lag_2,
    price_change_pct = (AveragePrice - price_lag_1) / price_lag_1 * 100,

    # Rolling statistics (4-week window)
    rolling_mean_4 = rollmean(AveragePrice, k = 4, fill = NA, align = "right"),
    rolling_sd_4 = rollapply(AveragePrice, width = 4, FUN = sd, fill = NA, align = "right"),
    rolling_max_4 = rollapply(AveragePrice, width = 4, FUN = max, fill = NA, align = "right"),
    rolling_min_4 = rollapply(AveragePrice, width = 4, FUN = min, fill = NA, align = "right"),

    # Year-over-year price
    price_yoy = AveragePrice - lag(AveragePrice, 52)
  ) %>%
  ungroup()

# Create interaction features
avocado <- avocado %>%
  mutate(
    type_region = paste(type, region, sep = "_"),
    volume_category = case_when(
      Total.Volume < quantile(Total.Volume, 0.25, na.rm = TRUE) ~ "Low",
      Total.Volume < quantile(Total.Volume, 0.75, na.rm = TRUE) ~ "Medium",
      TRUE ~ "High"
    )
  )

cat("Features created successfully.\n")

```

```
## Features created successfully.
```

```
cat("Total features:", ncol(avocado), "\n")
```

```
## Total features: 42
```

```
cat("\n===== CORRELATION ANALYSIS =====\n")
```

```
##
```

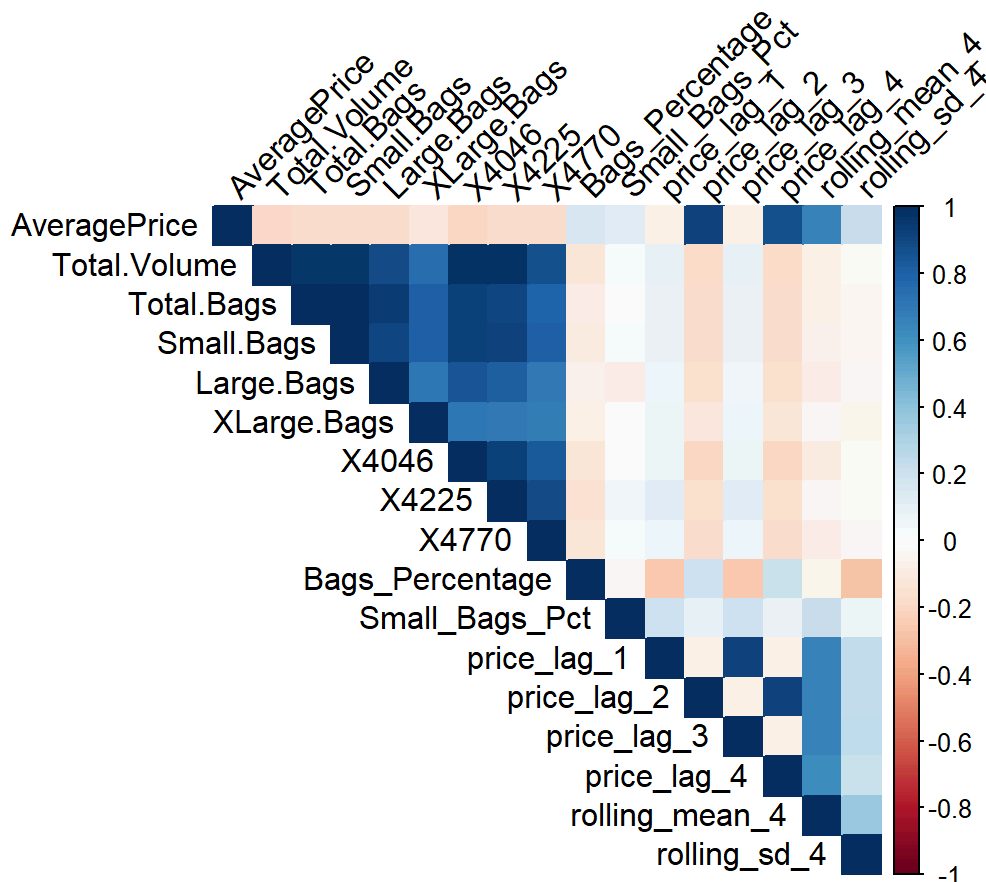
```
## ===== CORRELATION ANALYSIS =====
```

```
numeric_cols <- avocado %>%
  select(
    AveragePrice, Total.Volume, Total.Bags, Small.Bags, Large.Bags, XLarge.Bags,
    X4046, X4225, X4770, Bags_Percentage, Small_Bags_Pct,
    price_lag_1, price_lag_2, price_lag_3, price_lag_4,
    rolling_mean_4, rolling_sd_4
  ) %>%
  na.omit()

cor_matrix <- cor(numeric_cols, use = "complete.obs")

corrplot(cor_matrix,
  method = "color",
  type = "upper",
  tl.col = "black",
  tl.srt = 45,
  title = "Correlation Matrix of Avocado Features",
  mar = c(0,0,2,0))
```

Correlation Matrix of Avocado Features



```
highly_correlated <- which(abs(cor_matrix) > 0.8 & abs(cor_matrix) < 1, arr.ind = TRUE)
cat("\nHighly correlated feature pairs (>0.8):\n")
```

```
##
## Highly correlated feature pairs (>0.8):
```

```
if (length(highly_correlated) > 0) {
  for (i in 1:min(5, nrow(highly_correlated))) {
    row <- highly_correlated[i, 1]
    col <- highly_correlated[i, 2]
    if (row < col) {
      cat("  ", rownames(cor_matrix)[row], "&", colnames(cor_matrix)[col],
        ": ", round(cor_matrix[row, col], 3), "\n")
    }
  }
}
```

```
cat("\n===== TIME SERIES STATIONARITY =====\n")
```

```
##
## ===== TIME SERIES STATIONARITY =====
```

```
ca_ts <- avocado %>%
  filter(region == "California") %>%
  arrange(Date) %>%
  pull(AveragePrice) %>%
  ts(frequency = 52)

adf_test <- adf.test(ca_ts, alternative = "stationary")
cat("California prices - ADF Test p-value:", adf_test$p.value, "\n")
```

```
## California prices - ADF Test p-value: 0.2985419
```

```
cat("Stationary?", ifelse(adf_test$p.value < 0.05, "YES", "NO"), "\n")
```

```
## Stationary? NO
```

```
if (adf_test$p.value > 0.05) {
  cat("Series is NOT stationary. Will use differencing in ARIMA.\n")
}
```

```
## Series is NOT stationary. Will use differencing in ARIMA.
```

```
cat("\n===== DATA SPLITTING =====\n")
```

```
##
## ===== DATA SPLITTING =====
```

```
model_data <- avocado %>%
  select(
    Date,
    AveragePrice,
    Total.Volume,
    type,
    region,
    year,
    month,
    season,
    day_of_week,
    Bags_Percentage,
    Small_Bags_Pct,
    is_holiday_season,
    is_summer,
    price_lag_1,
    price_lag_2,
    price_lag_3,
    price_lag_4,
    rolling_mean_4,
    rolling_sd_4,
    price_change_1,
    price_yoy,
    is_outlier
  ) %>%
  na.omit()

cat("Model data dimensions:", dim(model_data), "\n")
```

```
## Model data dimensions: 15441 22
```

```
# Chronological split
set.seed(123)
n <- nrow(model_data)
train_end <- floor(n * 0.7)
valid_end <- floor(n * 0.85)

train_data <- model_data[1:train_end, ]
valid_data <- model_data[(train_end + 1):valid_end, ]
test_data <- model_data[(valid_end + 1):n, ]

cat("Training set:", nrow(train_data), "rows (", round(nrow(train_data)/n*100, 1), "%)\n")
```

```
## Training set: 10808 rows ( 70 %)
```

```
cat("Validation set:", nrow(valid_data), "rows (", round(nrow(valid_data)/n*100, 1), "%)\n")
```

```
## Validation set: 2316 rows ( 15 %)
```

```
cat("Test set:", nrow(test_data), "rows (", round(nrow(test_data)/n*100, 1), "%)\n")
```

```
## Test set: 2317 rows ( 15 %)
```

```
cat("Date ranges:\n")
```

```
## Date ranges:
```

```
cat("  Train:", format(min(train_data$Date), "%Y-%m-%d"), "to",  
    format(max(train_data$Date), "%Y-%m-%d"), "\n")
```

```
##   Train: 2015-07-05 to 2018-03-25
```

```
cat("  Valid:", format(min(valid_data$Date), "%Y-%m-%d"), "to",  
    format(max(valid_data$Date), "%Y-%m-%d"), "\n")
```

```
##   Valid: 2015-07-05 to 2018-03-25
```

```
cat("  Test:", format(min(test_data$Date), "%Y-%m-%d"), "to",  
    format(max(test_data$Date), "%Y-%m-%d"), "\n")
```

```
##   Test: 2015-07-05 to 2018-03-25
```

```
cat("\n===== DATA PREPROCESSING =====\n")
```

```
##
```

```
## ===== DATA PREPROCESSING =====
```

```
preprocess_steps <- preProcess(  
  train_data %>% select(where(is.numeric)),  
  method = c("center", "scale")  
)
```

```
train_scaled <- predict(preprocess_steps, train_data)  
valid_scaled <- predict(preprocess_steps, valid_data)  
test_scaled <- predict(preprocess_steps, test_data)
```

```
cat("Data preprocessing complete. Features scaled and centered.\n")
```

```
## Data preprocessing complete. Features scaled and centered.
```

```
# Save prepared data
saveRDS(list(
  train = train_data,
  valid = valid_data,
  test = test_data,
  train_scaled = train_scaled,
  valid_scaled = valid_scaled,
  test_scaled = test_scaled,
  scaler = preprocess_steps
), "avocado_prepared.rds")

cat("Prepared data saved to 'avocado_prepared.rds'\n")
```

```
## Prepared data saved to 'avocado_prepared.rds'
```

```
cat("📁 Location:", getwd(), "/avocado_prepared.rds\n")
```

```
## 📁 Location: C:/Users/Admin/Documents/project /avocado_prepared.rds
```

ARIMA TIME SERIES MODELING

```
# Fix avocado dates if needed
if(!inherits(avocado$Date, "Date") || min(avocado$Date, na.rm = TRUE) < 0) {
  avocado <- avocado %>%
    mutate(
      Date = as.Date(Date, origin = "1899-12-30")
    )
  cat("Fixed avocado dates\n")
}

region_name <- "California"
region_data <- avocado %>%
  filter(region == region_name) %>%
  arrange(Date) %>%
  mutate(month_name = month(Date, label = TRUE))

cat("Date range:", min(region_data$Date), "to", max(region_data$Date), "\n")
```

```
## Date range: 16439 to 17615
```

```
cat("Rows:", nrow(region_data), "\n")
```

```
## Rows: 338
```

```
# PHASE 3: ARIMA TIME SERIES MODELING
cat("\n===== \n")
```

```
##
## =====
```

```
cat("PHASE 3: ARIMA TIME SERIES MODELING \n")
```

```
## PHASE 3: ARIMA TIME SERIES MODELING
```

```
cat("===== \n \n")
```

```
## =====
```

```
# Prepare data
region_name <- "California"

region_data <- avocado %>%
  filter(region == region_name) %>%
  arrange(Date) %>%
  mutate(
    Date = as.Date(Date),
    month_name = month(Date, label = TRUE)
  )

cat("Data period:", min(region_data$Date), "to", max(region_data$Date), "\n")
```

```
## Data period: 16439 to 17615
```

```
cat("Number of weeks:", nrow(region_data), "\n")
```

```
## Number of weeks: 338
```

```
cat("Average price:", round(mean(region_data$AveragePrice), 3), "\n")
```

```
## Average price: 1.395
```



```
# Create time series
ts_data <- ts(
  region_data$AveragePrice,
  start = c(2015, 1),
  frequency = 52
)

cat("\nTime Series Object:\n")
```

```
##
## Time Series Object:
```

```
cat("  Start:", start(ts_data), "\n")
```

```
##    Start: 2015 1
```

```
cat("  End:", end(ts_data), "\n")
```

```
##    End: 2021 26
```

```
cat("  Frequency:", frequency(ts_data), "weeks/year\n")
```

```
##    Frequency: 52 weeks/year
```

```
cat("  Observations:", length(ts_data), "\n")
```

```
##    Observations: 338
```

```
# Stationarity test
cat("\n===== STATIONARITY TEST =====\n")
```

```
##
## ===== STATIONARITY TEST =====
```

```
adf_result <- adf.test(ts_data, alternative = "stationary")
cat("ADF Test p-value:", adf_result$p.value, "\n")
```

```
## ADF Test p-value: 0.2985419
```

```
cat("Is series stationary?", ifelse(adf_result$p.value < 0.05, "YES ✓", "NO X"), "\n")
```

```
## Is series stationary? NO X
```

```
# Train ARIMA (FAST VERSION)
cat("\n===== TRAINING ARIMA MODEL =====\n")
```

```
##
## ===== TRAINING ARIMA MODEL =====
```

```
n <- length(ts_data)
train_size <- floor(n * 0.8)
test_size <- n - train_size

train_ts <- ts_data[1:train_size]
test_ts <- ts_data[(train_size + 1):n]

cat("Training set:", train_size, "weeks\n")
```

```
## Training set: 270 weeks
```

```
cat("Test set:", test_size, "weeks\n")
```

```
## Test set: 68 weeks
```

```
set.seed(123)
cat("\nFinding best ARIMA parameters (FAST)...\n")
```

```
##
## Finding best ARIMA parameters (FAST)...
```

```
arima_model <- auto.arima(
  train_ts,
  seasonal = TRUE,
  stepwise = TRUE,
  approximation = TRUE,
  max.p = 3,
  max.q = 3,
  max.P = 1,
  max.Q = 1,
  trace = TRUE
)
```

```
##
## Fitting models using approximations to speed things up...
##
## ARIMA(2,1,2) with drift      : Inf
## ARIMA(0,1,0) with drift     : 487.2911
## ARIMA(1,1,0) with drift     : -397.288
## ARIMA(0,1,1) with drift     : 191.653
## ARIMA(0,1,0)                : 485.2765
## ARIMA(2,1,0) with drift     : Inf
## ARIMA(1,1,1) with drift     : Inf
## ARIMA(2,1,1) with drift     : Inf
## ARIMA(1,1,0)                : -398.8153
## ARIMA(2,1,0)                : Inf
## ARIMA(1,1,1)                : Inf
## ARIMA(0,1,1)                : 190.5126
## ARIMA(2,1,1)                : Inf
##
## Now re-fitting the best model(s) without approximations...
##
## ARIMA(1,1,0)                : -397.2466
##
## Best model: ARIMA(1,1,0)
```

```
cat("\n✅ Best ARIMA Model:\n")
```

```
##
## ✅ Best ARIMA Model:
```

```
print(arima_model)
```

```
## Series: train_ts
## ARIMA(1,1,0)
##
## Coefficients:
##          ar1
##        -0.9813
## s.e.      0.0108
##
## sigma^2 = 0.01306: log likelihood = 200.65
## AIC=-397.29   AICc=-397.25   BIC=-390.1
```

```
# Model diagnostics
residuals <- residuals(arima_model)
lb_test <- Box.test(residuals, lag = 10, type = "Ljung-Box")
cat("\nLjung-Box Test p-value:", lb_test$p.value, "\n")
```

```
##  
## Ljung-Box Test p-value: 0.01570185
```

```
cat("Residuals are", ifelse(lb_test$p.value > 0.05, "WHITE NOISE ✓", "CORRELATED X"), "\n")
```

```
## Residuals are CORRELATED X
```

```
# Forecast on test set  
forecast_result <- forecast(arma_model, h = test_size)  
forecast_mean <- as.numeric(forecast_result$mean)  
actual_test <- as.numeric(test_ts)  
  
rmse <- sqrt(mean((forecast_mean - actual_test)^2))  
mae <- mean(abs(forecast_mean - actual_test))  
mape <- mean(abs((forecast_mean - actual_test) / actual_test)) * 100  
  
cat("\nTest Set Performance:\n")
```

```
##  
## Test Set Performance:
```

```
cat(" RMSE:", round(rmse, 4), "\n")
```

```
## RMSE: 0.3331
```

```
cat(" MAE: ", round(mae, 4), "\n")
```

```
## MAE: 0.2636
```

```
cat(" MAPE:", round(mape, 2), "%\n")
```

```
## MAPE: 21.03 %
```

```
# 12-week forward forecast  
cat("\n===== 12-WEEK FORWARD FORECAST =====\n")
```

```
##  
## ===== 12-WEEK FORWARD FORECAST =====
```

```
final_model <- auto.arima(
  ts_data,
  seasonal = TRUE,
  stepwise = TRUE,
  approximation = TRUE,
  max.p = 3,
  max.q = 3,
  max.P = 1,
  max.Q = 1
)

forward_forecast <- forecast(final_model, h = 12)

last_date <- region_data$Date[nrow(region_data)]
if(!inherits(last_date, "Date")) {
  last_date <- as.Date(last_date)
}
if(is.null(last_date) || is.na(last_date)) {
  last_date <- max(region_data$Date, na.rm = TRUE)
}

forecast_dates <- seq.Date(
  from = as.Date(last_date) + 7,
  by = "week",
  length.out = 12
)

forecast_table <- data.frame(
  Date = forecast_dates,
  Forecast = round(as.numeric(forward_forecast$mean), 3),
  Lower_80 = round(as.numeric(forward_forecast$lower[, 1]), 3),
  Upper_80 = round(as.numeric(forward_forecast$upper[, 1]), 3),
  Lower_95 = round(as.numeric(forward_forecast$lower[, 2]), 3),
  Upper_95 = round(as.numeric(forward_forecast$upper[, 2]), 3)
)

cat("\n12-Week Price Forecast for", region_name, ":\n")
```

```
##
## 12-Week Price Forecast for California :
```

```
print(forecast_table)
```

##	Date	Forecast	Lower_80	Upper_80	Lower_95	Upper_95
## 1	2018-04-01	1.060	0.883	1.238	0.789	1.331
## 2	2018-04-08	1.682	1.497	1.867	1.399	1.965
## 3	2018-04-15	1.039	0.809	1.268	0.688	1.389
## 4	2018-04-22	1.728	1.491	1.964	1.365	2.090
## 5	2018-04-29	1.166	0.902	1.430	0.762	1.570
## 6	2018-05-06	1.734	1.464	2.004	1.321	2.147
## 7	2018-05-13	1.086	0.796	1.375	0.642	1.529
## 8	2018-05-20	1.663	1.369	1.958	1.214	2.113
## 9	2018-05-27	0.964	0.655	1.273	0.491	1.437
## 10	2018-06-03	1.685	1.372	1.997	1.207	2.163
## 11	2018-06-10	0.870	0.546	1.194	0.374	1.366
## 12	2018-06-17	1.576	1.249	1.903	1.076	2.076

```
# =====
# PHASE 3B: MACHINE LEARNING MODELS (RANDOM FOREST)
# =====

cat("\n=====\\n")
```

```
##
## =====
```

```
cat("PHASE 3B: MACHINE LEARNING MODELS\\n")
```

```
## PHASE 3B: MACHINE LEARNING MODELS
```

```
cat("=====\\n\\n")
```

```
## =====
```

```
# Prepare data for ML
ml_data <- model_data %>%
  select(-Date) %>%
  na.omit()

cat("ML Data dimensions:", dim(ml_data), "\\n")
```

```
## ML Data dimensions: 15441 21
```

```
cat("Target variable: AveragePrice\\n")
```

```
## Target variable: AveragePrice
```

```
cat("Features:", ncol(ml_data) - 1, "\n")
```

```
## Features: 20
```

```
# Prepare ML datasets
train_ml <- train_data %>% select(-Date) %>% na.omit()
valid_ml <- valid_data %>% select(-Date) %>% na.omit()
test_ml <- test_data %>% select(-Date) %>% na.omit()

cat("\nTraining set:", nrow(train_ml), "rows\n")
```

```
##
## Training set: 10808 rows
```

```
cat("Validation set:", nrow(valid_ml), "rows\n")
```

```
## Validation set: 2316 rows
```

```
cat("Test set:", nrow(test_ml), "rows\n")
```

```
## Test set: 2317 rows
```

```
# Train Random Forest
cat("\n===== TRAINING RANDOM FOREST =====\n")
```

```
##
## ===== TRAINING RANDOM FOREST =====
```

```
set.seed(123)
rf_model <- randomForest(
  AveragePrice ~ .,
  data = train_ml,
  ntree = 100,
  importance = TRUE,
  mtry = floor(sqrt(ncol(train_ml) - 1)),
  nodesize = 5
)

cat("✅ Random Forest trained!\n")
```

```
## ✅ Random Forest trained!
```

```
print(rf_model)
```

```
##
## Call:
## randomForest(formula = AveragePrice ~ ., data = train_ml, ntree = 100, importance = TRUE, mtry = floor(sqrt(ncol(train_ml) - 1)), nodesize = 5)
##           Type of random forest: regression
##           Number of trees: 100
## No. of variables tried at each split: 4
##
##           Mean of squared residuals: 0.004049291
##           % Var explained: 97.43
```

```
# Random Forest Evaluation
cat("\n===== RANDOM FOREST EVALUATION =====\n")
```

```
##
## ===== RANDOM FOREST EVALUATION =====
```

```
rf_pred <- predict(rf_model, newdata = test_ml)

rf_rmse <- sqrt(mean((rf_pred - test_ml$AveragePrice)^2))
rf_mae <- mean(abs(rf_pred - test_ml$AveragePrice))
rf_r2 <- cor(rf_pred, test_ml$AveragePrice)^2
rf_mape <- mean(abs((rf_pred - test_ml$AveragePrice) / test_ml$AveragePrice)) * 100

cat("Random Forest Performance:\n")
```

```
## Random Forest Performance:
```

```
cat("  RMSE:", round(rf_rmse, 4), "\n")
```

```
##    RMSE: 0.0629
```

```
cat("  MAE: ", round(rf_mae, 4), "\n")
```

```
##    MAE:  0.0453
```

```
cat("  R²:  ", round(rf_r2, 4), "\n")
```

```
##    R²:   0.977
```

```
cat("  MAPE:", round(rf_mape, 2), "%\n")
```

```
##    MAPE: 3.47 %
```



```
# Feature Importance
importance_df <- as.data.frame(importance(rf_model))
importance_df$Feature <- rownames(importance_df)
importance_df <- importance_df %>%
  arrange(desc(`%IncMSE`))

cat("\nTop 10 Important Features:\n")
```

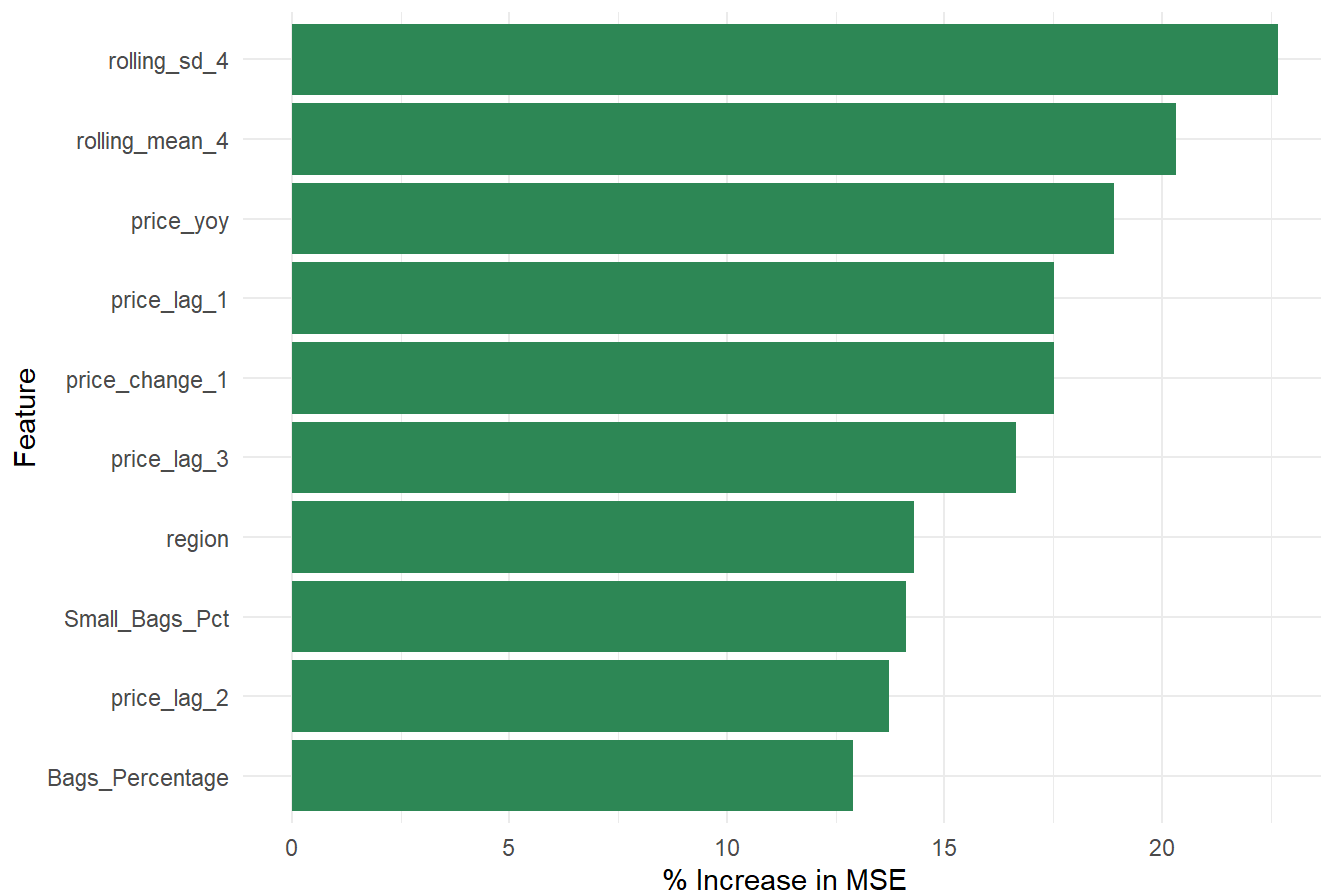
```
##
## Top 10 Important Features:
```

```
print(head(importance_df, 10))
```

```
##           %IncMSE IncNodePurity      Feature
## rolling_sd_4    22.66075      47.10849  rolling_sd_4
## rolling_mean_4  20.31701     232.32952  rolling_mean_4
## price_yoy       18.90741      63.98862   price_yoy
## price_lag_1     17.51493      50.85915   price_lag_1
## price_change_1  17.50988     289.49745  price_change_1
## price_lag_3     16.64354      34.03211   price_lag_3
## region          14.30851       7.03023    region
## Small_Bags_Pct  14.11934      16.67242  Small_Bags_Pct
## price_lag_2     13.72932     461.74594   price_lag_2
## Bags_Percentage 12.90221      16.83265  Bags_Percentage
```

```
# Feature importance plot
imp_plot <- ggplot(head(importance_df, 10),
  aes(x = reorder(Feature, `%IncMSE`), y = `%IncMSE`) +
  geom_col(fill = "#2E8B57") +
  coord_flip() +
  labs(
    title = "Random Forest Feature Importance",
    x = "Feature",
    y = "% Increase in MSE"
  ) +
  theme_minimal()
print(imp_plot)
```

Random Forest Feature Importance



```
# Actual vs Predicted plot
rf_results <- data.frame(
  Actual = test_ml$AveragePrice,
  Predicted = rf_pred
)

rf_plot <- ggplot(rf_results, aes(x = Actual, y = Predicted)) +
  geom_point(alpha = 0.4, color = "#2E8B57") +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +
  labs(
    title = paste("Random Forest: Actual vs Predicted (R² =", round(rf_r2, 3), ")"),
    x = "Actual Price (USD)",
    y = "Predicted Price (USD)"
  ) +
  theme_minimal()
print(rf_plot)
```

Random Forest: Actual vs Predicted ($R^2 = 0.977$)

```
# Model Comparison
cat("\n===== MODEL COMPARISON =====\n")
```

```
##
## ===== MODEL COMPARISON =====
```

```
comparison_df <- data.frame(
  Model = c("ARIMA", "Random Forest"),
  RMSE = c(round(rmse, 4), round(rf_rmse, 4)),
  MAPE = c(round(mape, 2), round(rf_mape, 2)),
  R2 = c(NA, round(rf_r2, 4))
)

cat("Model Comparison:\n")
```

```
## Model Comparison:
```

```
print(comparison_df)
```

```
##           Model  RMSE  MAPE    R2
## 1           ARIMA 0.3331 21.03   NA
## 2 Random Forest 0.0629  3.47 0.977
```

```
if (rf_rmse < rmse) {
  cat("\n✅ Random Forest performs better (lower RMSE)\n")
} else {
  cat("\n✅ ARIMA performs better (lower RMSE)\n")
}
```

```
##
## ✅ Random Forest performs better (lower RMSE)
```

```
# Save ML model
saveRDS(rf_model, "rf_model.rds")
cat("\n💾 Random Forest model saved to: rf_model.rds\n")
```

```
##
## 💾 Random Forest model saved to: rf_model.rds
```

```
cat("\n===== \n")
```

```
##
## =====
```

```
cat("PHASE 4: ARIMA EVALUATION\n")
```

```
## PHASE 4: ARIMA EVALUATION
```

```
cat("===== \n\n")
```

```
## =====
```

```
cat("===== MODEL PERFORMANCE ===== \n")
```

```
## ===== MODEL PERFORMANCE =====
```

```
cat("Model:", as.character(arma_model), "\n")
```

```
## Model: ARIMA(1,1,0)
```

```
cat("Training data:", train_size, "weeks\n")
```

```
## Training data: 270 weeks
```

```
cat("Test data:", test_size, "weeks\n")
```

```
## Test data: 68 weeks
```

```
cat("\nAccuracy Metrics:\n")
```

```
##  
## Accuracy Metrics:
```

```
cat("  RMSE:", round(rmse, 4), "\n")
```

```
##  RMSE: 0.3331
```

```
cat("  MAE: ", round(mae, 4), "\n")
```

```
##  MAE:  0.2636
```

```
cat("  MAPE:", round(mape, 2), "%\n")
```

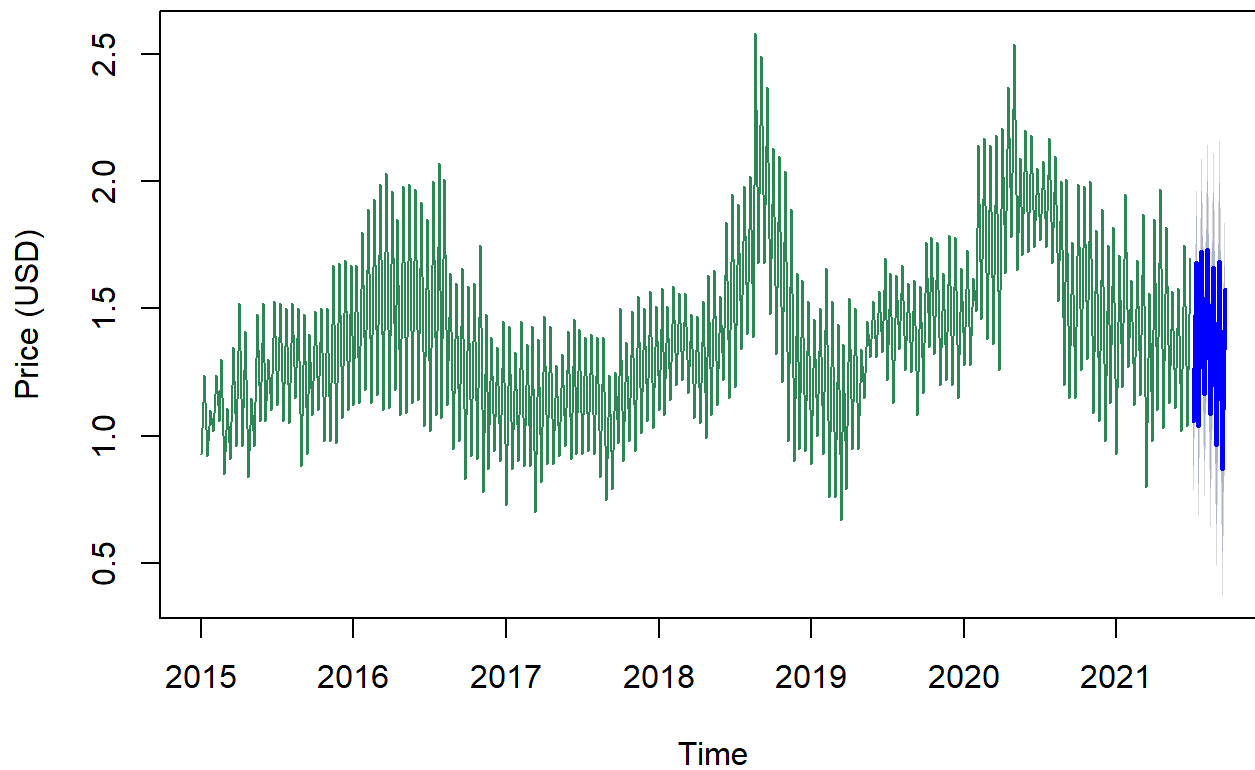
```
##  MAPE: 21.03 %
```

```
if (mape < 5) {  
  cat("  ✅ Excellent forecast accuracy (<5% error)\n")  
} else if (mape < 10) {  
  cat("  ✅ Good forecast accuracy (5-10% error)\n")  
} else if (mape < 15) {  
  cat("  ⚠️ Moderate forecast accuracy (10-15% error)\n")  
} else {  
  cat("  ⚠️ Poor forecast accuracy (>15% error)\n")  
}
```

```
##  ⚠️ Poor forecast accuracy (>15% error)
```

```
# Forecast plot
plot(forward_forecast,
     main = paste("12-Week Forecast -", region_name),
     xlab = "Time",
     ylab = "Price (USD)",
     col = "#2E8B57",
     fcol = "blue",
     flwd = 2)
```

12-Week Forecast - California



```
cat("\n===== 12-WEEK FORECAST =====\n")
```

```
##
## ===== 12-WEEK FORECAST =====
```

```
print(forecast_table)
```

##	Date	Forecast	Lower_80	Upper_80	Lower_95	Upper_95
## 1	2018-04-01	1.060	0.883	1.238	0.789	1.331
## 2	2018-04-08	1.682	1.497	1.867	1.399	1.965
## 3	2018-04-15	1.039	0.809	1.268	0.688	1.389
## 4	2018-04-22	1.728	1.491	1.964	1.365	2.090
## 5	2018-04-29	1.166	0.902	1.430	0.762	1.570
## 6	2018-05-06	1.734	1.464	2.004	1.321	2.147
## 7	2018-05-13	1.086	0.796	1.375	0.642	1.529
## 8	2018-05-20	1.663	1.369	1.958	1.214	2.113
## 9	2018-05-27	0.964	0.655	1.273	0.491	1.437
## 10	2018-06-03	1.685	1.372	1.997	1.207	2.163
## 11	2018-06-10	0.870	0.546	1.194	0.374	1.366
## 12	2018-06-17	1.576	1.249	1.903	1.076	2.076

```
# Seasonal Insights
```

```
cat("\n===== SEASONAL INSIGHTS =====\n")
```

```
##
```

```
## ===== SEASONAL INSIGHTS =====
```

```
# Create seasonal summary
```

```
seasonal_summary <- region_data %>%
```

```
  group_by(month_name) %>%
```

```
  summarise(
```

```
    avg_price = mean(AveragePrice),
```

```
    min_price = min(AveragePrice),
```

```
    max_price = max(AveragePrice)
```

```
  ) %>%
```

```
  arrange(desc(avg_price))
```

```
cat("\nAverage price by month:\n")
```

```
##
```

```
## Average price by month:
```

```
print(seasonal_summary)
```

```
## # A tibble: 12 × 4
##   month_name avg_price min_price max_price
##   <ord>      <dbl>    <dbl>    <dbl>
## 1 Oct         1.72      1.02      2.58
## 2 Sep         1.63      1.04      2.2
## 3 Aug         1.58      0.99      2.54
## 4 Nov         1.53      0.83      2.37
## 5 Jul         1.50      1.08      2.17
## 6 Jun         1.36      0.94      1.79
## 7 Apr         1.31      0.84      1.67
## 8 Dec         1.30      0.78      1.89
## 9 Mar         1.28      0.84      1.75
## 10 May        1.26      0.75      1.78
## 11 Jan        1.22      0.73      1.95
## 12 Feb        1.19      0.67      1.97
```

```
best_month <- seasonal_summary$month_name[which.min(seasonal_summary$avg_price)]
worst_month <- seasonal_summary$month_name[which.max(seasonal_summary$avg_price)]

cat("\n 📊 Seasonal Insights:\n")
```

```
##
## 📊 Seasonal Insights:
```

```
cat(" Best month to buy (lowest price):", best_month, "\n")
```

```
## Best month to buy (lowest price): 2
```

```
cat(" Worst month to buy (highest price):", worst_month, "\n")
```

```
## Worst month to buy (highest price): 10
```

```
cat(" Price difference: $",
    round(max(seasonal_summary$avg_price) - min(seasonal_summary$avg_price), 3), "\n")
```

```
## Price difference: $ 0.532
```

```
# Business Insights
cat("\n===== BUSINESS INSIGHTS =====\n")
```

```
##
## ===== BUSINESS INSIGHTS =====
```



```
price_trend <- ifelse(
  mean(tail(region_data$AveragePrice, 4)) > mean(head(region_data$AveragePrice, 4)),
  "UPWARD", "DOWNWARD"
)

cat(" 📊 Market Trends:\n")
```

```
## 📊 Market Trends:
```

```
cat("  Current price trend:", price_trend, "\n")
```

```
##   Current price trend: UPWARD
```

```
cat("  Current average price: $", round(mean(tail(region_data$AveragePrice, 4)), 3), "\n")
```

```
##   Current average price: $ 1.378
```

```
cat("  Predicted price in 4 weeks: $", forecast_table$Forecast[4], "\n")
```

```
##   Predicted price in 4 weeks: $ 1.728
```

```
cat("  Predicted price in 12 weeks: $", forecast_table$Forecast[12], "\n")
```

```
##   Predicted price in 12 weeks: $ 1.576
```

```
price_change_4w <- ((forecast_table$Forecast[4] - mean(tail(region_data$AveragePrice, 4))) /
  mean(tail(region_data$AveragePrice, 4))) * 100

if (price_change_4w < -5) {
  cat(" 💡 Recommendation: BUY NOW - prices expected to drop\n")
} else if (price_change_4w > 5) {
  cat(" 💡 Recommendation: SELL NOW - prices expected to rise\n")
} else {
  cat(" 💡 Recommendation: HOLD - prices stable\n")
}
```

```
##   💡 Recommendation: SELL NOW - prices expected to rise
```

```
cat("\n===== \n")
```

```
##
## =====
```

```
cat("✅ PHASE 4 COMPLETE!\n")
```

```
## ✅ PHASE 4 COMPLETE!
```

```
cat("=====\n")
```

```
## =====
```

```
# =====  
# DEPLOYMENT - RUN FROM RMD  
# =====  
  
# Save models  
saveRDS(final_model, "arima_model.rds")  
saveRDS(rf_model, "rf_model.rds")  
  
# Save results  
write.csv(forecast_table, "forecast_results.csv", row.names = FALSE)  
write.csv(comparison_df, "model_comparison.csv", row.names = FALSE)  
  
cat("✅ Deployment files created!\n")
```

```
## ✅ Deployment files created!
```

```
cat("📁 Files: arima_model.rds, rf_model.rds, forecast_results.csv\n")
```

```
## 📁 Files: arima_model.rds, rf_model.rds, forecast_results.csv
```